

Local Text-to-SQL Model Comparison

CS496 AI Engineering • Project 0 • Final test results • 14 September 2026

1. Task

We compare four locally hosted models that translate ecommerce questions into SQLite queries. The generated and reference SQL are executed on the same database; their returned rows, not SQL strings, determine correctness. Row order is compared only when requested; column values, column order and duplicate rows still matter. Failures count as incorrect.

Example: “How many customers live in Tunisia?”

Reference: `SELECT COUNT(*) FROM customers WHERE country = 'Tunisia';`

2. Dataset

The synthetic shop database uses seed 42: customers(id, name, country), 300 rows; products(id, name, category, price), 20 rows; orders(id, customer_id, product_id, quantity, order_date), 1,200 rows. Orders reference customer and product IDs; revenue means price × quantity.

There are 10 development items and 50 final-test items (20 easy, 22 medium, 8 hard). Development items supported prompt/parser debugging; the final set stayed unchanged across these test runs.

3. Experimental setup

Models: gemma3:1b, llama3.2:3b, phi4-mini (latest), qwen3:8b. Each receives the same schema, prompt rules, question order and shared parser (plain SQL, sql/sqlite/unlabeled fences). Settings: temperature 0, output limit 256 tokens, 60-second client timeout; Qwen thinking disabled. Runtime logs confirm 4,096-token contexts for all four.

Final hardware: EndeavourOS Linux; Intel i7-10870H (8 cores/16 threads), 16 GB RAM, NVIDIA GTX 1650 (4 GiB VRAM), Ollama 0.33.3. GPU-offloaded layers: Gemma 27/27, Llama 29/29, Phi 28/33, Qwen 16/37; Phi/Qwen used CPU and GPU. A teammate’s earlier Qwen dev run used Ryzen 7 5700X / RTX 3060 / 31.9 GB RAM on Windows; its measurements are excluded.

Each model processed 50 requests sequentially. No separate warm-up was used; latency includes loading when needed.

4. Results

Model	Correct/50 (%)	p50 ms	p95 ms	Tokens/s	€/1K
gemma3:1b	21/50 (42%)	898.73	1,273.66	108.10	0.0522
llama3.2:3b	32/50 (64%)	670.14	1,497.22	58.95	0.0477
phi4-mini	50/50 (100%)	1,642.44	3,901.67	31.67	0.1044
qwen3:8b	47/50 (94%)	5,141.19	12,683.48	7.97	0.3329

5. Wrong-answer examples

Three saved generated queries per model where available. Item IDs refer to data/database/items.jsonl (test split).

gemma3:1b

#7 — Lowest catalog price

```
SELECT p.price FROM products p WHERE p.category = 'All';
```

#11 — Total quantity ordered in 2024

```
SELECT SUM(quantity) FROM orders WHERE order_date LIKE '2024/01%';
```

#19 — Average quantity per order

```
SELECT SUM(quantity) / COUNT(*) FROM orders;
```

llama3.2:3b

#6 — Average Furniture price

```
SELECT AVG(T2.price) FROM products AS T1 INNER JOIN orders AS T2 ON T1.id = T2.product_id WHERE T1.category = 'Furniture';
```

#11 — Total quantity ordered in 2024

```
SELECT SUM(quantity) FROM orders WHERE order_date LIKE '%/%/24';
```

#15 — Highest-priced product; smallest ID breaks ties

```
SELECT name, price FROM products ORDER BY price DESC, id ASC;
```

qwen3:8b

#44 — Products priced above their own category average

```
SELECT id, name, category, price FROM products WHERE price > ( SELECT AVG(price) FROM products AS p WHERE p.category = category );
```

#45 — Highest-revenue country in 2024; alphabetical tie-break

```
SELECT country, SUM(products.price * orders.quantity) AS total_revenue FROM orders JOIN products ON orders.product_id = products.id JOIN customers ON orders.customer_id = customers.id WHERE order_date LIKE '2024%' GROUP BY country ORDER BY total_revenue DESC, country ASC;
```

#50 — Customers ordering in 2024 but not 2025

```
SELECT customers.id, customers.name FROM customers JOIN orders ON customers.id = orders.customer_id WHERE orders.order_date BETWEEN '2024-01-01' AND '2024-12-31' AND customers.id NOT IN ( SELECT DISTINCT orders.customer_id FROM orders WHERE orders.order_date BETWEEN '2025-01-01' AND '2025-12-31' );
```

phi4-mini: 50/50 correct; no wrong answers in the final test run.

6. Cost and scaling

Assumed hardware operating cost: €0.20/hour for every model. Hardware-only cost per 1,000 requests = $(0.20 \div \text{measured requests/hour}) \times 1,000$. This is a cost scenario, not metered expenditure. Requests/hour includes the whole evaluation loop, including SQL scoring and Python overhead.

Model	Requests/hour	Assumed €/hour
gemma3:1b	3,833.13	0.20
llama3.2:3b	4,191.72	0.20
phi4-mini	1,916.33	0.20
qwen3:8b	600.79	0.20

7. Final choice

We select phi4-mini for this workload: 50/50 correct at 1,642.44 ms p50, 3,901.67 ms p95 and €0.1044/1K. Qwen achieved 47/50 with higher latency and cost. Llama is faster and cheaper (64%, €0.0477/1K), but its error rate is too high for our accuracy-first use case. We would reconsider if p95 below 1.5 seconds became mandatory and 64% accuracy were acceptable, or if a larger unseen dataset or different hardware changed the ranking. Phi's 100% applies only to these 50 test items.